

Análisis de Comunidades ciclistas en el AMBA

El siguiente cuaderno tiene como objetivo analizar los chat grupales y las redes sociales de 4 comunidades ciclistas activas:

- Hormiguita Viajera
- Salidas de Cicloturismo
- Los caidos
- Bicinaturas

Configuración inicial

```
!pip install community
```

```
Requirement already satisfied: community in c:\users\sebas\anaconda3\lib\site-packages (1.0.0b1)
Requirement already satisfied: Flask in c:\users\sebas\anaconda3\lib\site-packages (from community) (2.2.5)
Requirement already satisfied: requests in c:\users\sebas\anaconda3\lib\site-packages (from community) (2.31.0)
Requirement already satisfied: Werkzeug>=2.2.2 in c:\users\sebas\anaconda3\lib\site-packages (from Flask->community) (2.2.3)
Requirement already satisfied: Jinja2>=3.0 in c:\users\sebas\anaconda3\lib\site-packages (from Flask->community) (3.1.3)
Requirement already satisfied: itsdangerous>=2.0 in c:\users\sebas\anaconda3\lib\site-packages (from Flask->community) (2.0.1)
Requirement already satisfied: click>=8.0 in c:\users\sebas\anaconda3\lib\site-packages (from Flask->community) (8.1.7)
Requirement already satisfied: charset-normalizer<4,>=2 in c:\users\sebas\anaconda3\lib\site-packages (from requests->community) (2.0.4)
Requirement already satisfied: idna<4,>=2.5 in c:\users\sebas\anaconda3\lib\site-packages (from requests->community) (3.4)
Requirement already satisfied: urllib3<3,>=1.21.1 in c:\users\sebas\anaconda3\lib\site-packages (from requests->community) (2.0.7)
Requirement already satisfied: certifi>=2017.4.17 in c:\users\sebas\anaconda3\lib\site-packages (from requests->community) (2024.2.2)
Requirement already satisfied: colorama in c:\users\sebas\anaconda3\lib\site-packages (from click>=8.0->Flask->community) (0.4.6)
Requirement already satisfied: MarkupSafe>=2.0 in c:\users\sebas\anaconda3\lib\site-packages (from Jinja2>=3.0->Flask->community) (2.1.3)
```

```
import pandas as pd
import networkx as nx
import matplotlib.pyplot as plt
import seaborn as sns
import re
```

```

from datetime import datetime, timedelta
import warnings
import hashlib

```

```
warnings.filterwarnings('ignore')
```

```
# Configuración de visualización
```

```

sns.set(style="whitegrid")
plt.rcParams['figure.figsize'] = (12, 8)

```

Función de Procesamiento de Chats (ETL)

```
def get_anonymous_id(identifier):
```

```

    """
    Convierte un nombre/teléfono en un ID único y consistente de 6
    caracteres.

```

```

    Ejemplo: '+54 9 11 1234-5678' -> 'U_a1b2c'
    """

```

```

    # Usamos SHA-256 para crear una huella digital del texto

```

```
    hash_object = hashlib.sha256(identifier.encode('utf-8'))
```

```

    # Tomamos solo los primeros 6 caracteres para que el gráfico no se
    llene de texto largo

```

```
    short_hash = hash_object.hexdigest()[:6]
```

```
    return f"U_{short_hash}"
```

```
def parse_whatsapp_chat(file_path, community_name, anonymize=True):
```

```

    with open(file_path, 'r', encoding='utf-8') as f:

```

```
        content = f.read()
```

```

    # Regex estándar para WhatsApp

```

```

    pattern = r'^(\d{1,2}/\d{1,2}/\d{4}), (\d{2}:\d{2}) - (.*?): (.*'
    $'

```

```
    data = []
```

```

    for line in content.split('\n'):

```

```
        match = re.match(pattern, line)
```

```
        if match:
```

```
            date_str, time_str, sender, message = match.groups()
```

```

            dt = datetime.strptime(f"{date_str} {time_str}", "%d/%m/%Y
%H:%M")

```

```

            # Limpieza básica

```

```

            sender = sender.strip().replace('\u202c', '').replace('\
u202a', '')

```

```

            # --- AQUÍ OCURRE LA MAGIA DE LA ANONIMIZACIÓN ---

```

```
            if anonymize:
```

```

                # Excluimos mensajes del sistema si es necesario, o
                los dejamos pasar

```

```

        if "Cambió tu código" not in message:
            sender = get_anonymous_id(sender)
# -----

    data.append({
        'datetime': dt,
        'sender': sender,
        'message': message,
        'community': community_name
    })

    df = pd.DataFrame(data)
    print(f"Procesado '{community_name}': {len(df)} mensajes.")
    return df

files_map = {
    'Data/Chat de Whatsapp con Bicinautas.txt': 'Bicinautas',
    'Data/Chat de WhatsApp con Los caidos salidas.txt': 'Los Caídos',
    'Data/Chat de WhatsApp con Social Difusion HORMIGUITA VIAJERA.txt': 'Hormiguita Viajera',
    'Data/Chat de WhatsApp con Salidas de cicloturismo.txt': 'Salidas Cicloturismo'
}

dfs = []
for filename, community_name in files_map.items():
    try:
        # Note que 'anonymize=True' está activado por defecto
        df_temp = parse_whatsapp_chat(filename, community_name,
anonymize=True)
        dfs.append(df_temp)
    except FileNotFoundError:
        print(f"⚠ Archivo no encontrado: {filename}")

Procesado 'Bicinautas': 98538 mensajes.
Procesado 'Los Caídos': 832 mensajes.
Procesado 'Hormiguita Viajera': 29779 mensajes.
Procesado 'Salidas Cicloturismo': 19383 mensajes.

if dfs:
    all_chats = pd.concat(dfs, ignore_index=True)
    display(all_chats.head())

```

	datetime	sender	\
0	2024-08-22 16:10:00	U_d343b8	
1	2024-08-22 16:10:00	U_d343b8	
2	2024-08-22 16:11:00	U_d343b8	
3	2024-08-22 16:15:00	U_6996d9	
4	2024-08-22 16:15:00	U_d6fc0e	


```

    return G

# Construimos la red
G = build_interaction_network(all_chats, time_window_minutes=10) #
Ventana de 10 mins

print(f"La Red Social Ciclista construida tiene {G.number_of_nodes()}
ciclistas y {G.number_of_edges()} interacciones.")

La Red Social Ciclista construida tiene 1585 ciclistas y 23064
interacciones.

```

Análisis de Homofilia

```

def analyze_homophily(G):
    """
    Analiza la homofilia basada en la comunidad de origen.
    Calcula el coeficiente de asortatividad nominal.
    """
    # Asortatividad: Medida de -1 a 1.
    # Positivo alto = Alta Homofilia (se mezclan solo con los suyos).
    # Cero o negativo = Heterofilia (se mezclan aleatoriamente o con
    distintos).

    r = nx.attribute_assortativity_coefficient(G, 'community_label')

    print(f"Coeficiente de Homofilia (Asortatividad por Comunidad):
    {r:.4f}")

    if r > 0.1:
        print("INTERPRETACIÓN: Existe una tendencia a la Homofilia.
        Los ciclistas interactúan principalmente dentro de sus grupos
        originales.")
    elif r < -0.1:
        print("INTERPRETACIÓN: Existe Heterofilia. Hay mucha mezcla
        entre grupos.")
    else:
        print("INTERPRETACIÓN: No hay un patrón fuerte de homofilia
        basado en el grupo de origen.")

analyze_homophily(G)

Coeficiente de Homofilia (Asortatividad por Comunidad): 0.5397
INTERPRETACIÓN: Existe una tendencia a la Homofilia. Los ciclistas
interactúan principalmente dentro de sus grupos originales.

```

Detección de Lazos Débiles y Puentes (Granovetter)

```
def find_bridges_and_weak_ties(all_chats):  
    """  
    Identifica usuarios que aparecen en múltiples archivos de chat.  
    Estos usuarios actúan como puentes de información (Lazos Débiles  
    intergrupales).  
    """  
    # Tabla pivote: Usuario vs Comunidad  
    user_presence = all_chats.groupby(['sender',  
    'community']).size().unstack(fill_value=0)  
  
    # Convertir a 1 si está presente, 0 si no  
    user_presence_binary = (user_presence > 0).astype(int)  
  
    # Sumar columnas para ver en cuántas comunidades participa  
    user_presence_binary['total_comunidades'] =  
    user_presence_binary.sum(axis=1)  
  
    # Filtrar los "Multicomunitarios" (Puentes)  
    bridges =  
    user_presence_binary[user_presence_binary['total_comunidades'] > 1]  
  
    print(f"Se detectaron {len(bridges)} usuarios que actúan como  
    PUENTES entre comunidades.")  
    display(bridges.sort_values('total_comunidades', ascending=False))  
  
    return bridges
```

```
bridges_df = find_bridges_and_weak_ties(all_chats)
```

Se detectaron 144 usuarios que actúan como PUENTES entre comunidades.

```
community  Bicinautas  Hormiguita Viajera  Los Caídos  Salidas  
Cicloturismo \  
sender
```

U_78aa37	1	1	0
1			
U_17095f	1	1	0
1			
U_629430	1	1	0
1			
U_659ac0	1	1	0
1			
U_457a35	1	1	0
1			
...	...	...	...
...			
U_602ef8	1	1	0

0			
U_5f286e	1	1	0
0			
U_5bacfc	1	1	0
0			
U_5a6ab2	1	1	0
0			
U_f8f1df	1	1	0
0			

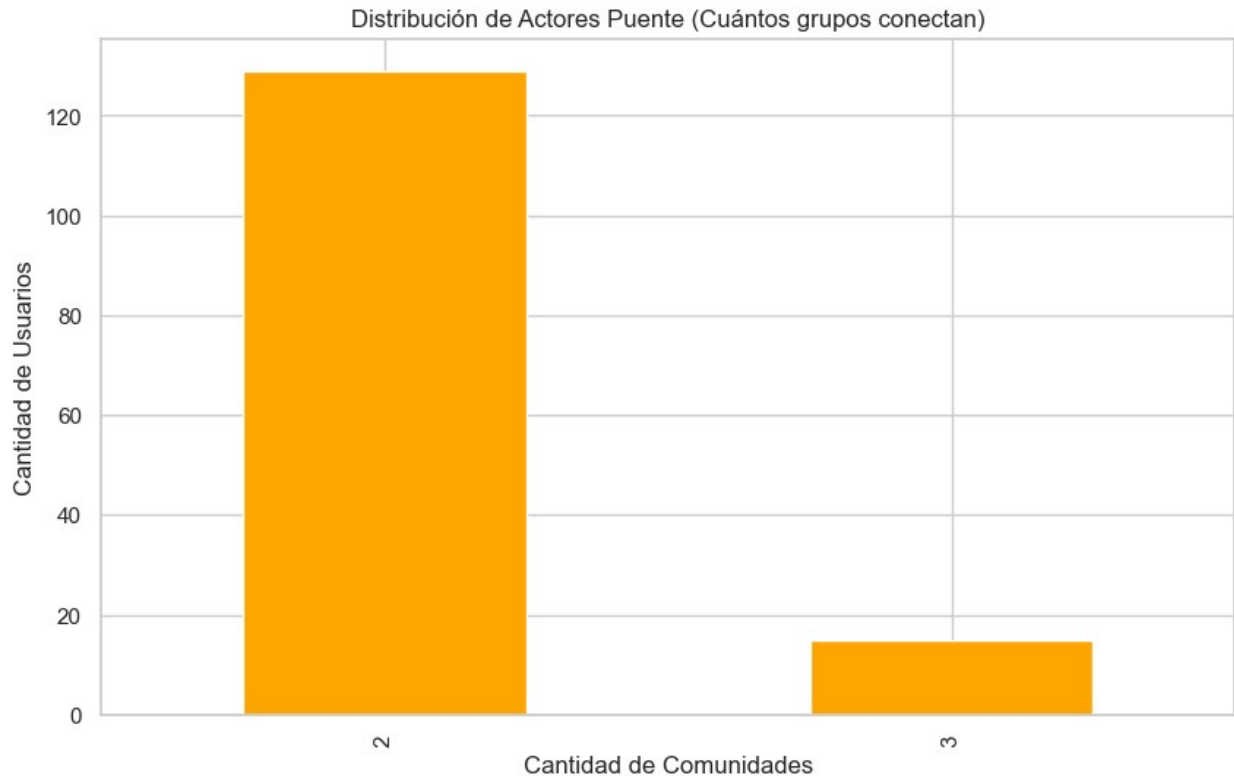
community	total_comunidades
sender	
U_78aa37	3
U_17095f	3
U_629430	3
U_659ac0	3
U_457a35	3
...	...
U_602ef8	2
U_5f286e	2
U_5bacfc	2
U_5a6ab2	2
U_f8f1df	2

[144 rows x 5 columns]

Visualización simple de los puentes

```
if not bridges_df.empty:
    plt.figure(figsize=(10, 6))
```

```
bridges_df['total_comunidades'].value_counts().sort_index().plot(kind=
'bar', color='orange')
plt.title("Distribución de Actores Puente (Cuántos grupos
conectan)")
plt.xlabel("Cantidad de Comunidades")
plt.ylabel("Cantidad de Usuarios")
plt.show()
```



Métricas de Centralidad (Líderes vs. Intermediarios)

```
# Calcular centralidades
degree_cent = nx.degree centrality(G)
betweenness_cent = nx.betweenness centrality(G)

# Crear DataFrame con métricas
metrics_df = pd.DataFrame({
    'Grado (Popularidad)': degree_cent,
    'Intermediación (Control Info)': betweenness_cent
})
```

```
# Top 10 Influencers (Líderes de opinión)
print("TOP 10 - Actores Centrales (Mayor actividad):")
display(metrics_df.sort_values('Grado (Popularidad)',
    ascending=False).head(10))
```

TOP 10 - Actores Centrales (Mayor actividad):

	Grado (Popularidad)	Intermediación (Control Info)
U_8b0c27	0.715278	0.242042
U_d44266	0.390152	0.122559
U_af4b77	0.356692	0.047538
U_ef63eb	0.314394	0.040108
U_17c8cd	0.309975	0.063696
U_ea0917	0.236742	0.020372

U_92ace0	0.221591	0.015021
U_59d95b	0.217803	0.058653
U_8c18c4	0.213384	0.013651
U_2a2555	0.205177	0.065480

```
# Top 10 Puentes Estructurales (Lazos débiles estratégicos)
print("\nTOP 10 - Actores Intermediarios (Capacidad de conectar subgrupos):")
display(metrics_df.sort_values('Intermediación (Control Info)', ascending=False).head(10))
```

TOP 10 - Actores Intermediarios (Capacidad de conectar subgrupos):

	Grado (Popularidad)	Intermediación (Control Info)
U_8b0c27	0.715278	0.242042
U_d44266	0.390152	0.122559
U_2a2555	0.205177	0.065480
U_17c8cd	0.309975	0.063696
U_792cf8	0.202652	0.060573
U_59d95b	0.217803	0.058653
U_dda550	0.166035	0.047875
U_af4b77	0.356692	0.047538
U_c4db79	0.130682	0.040751
U_ef63eb	0.314394	0.040108

Visualización de la Red (Grafo)

```
def plot_network_improved(G):
    plt.figure(figsize=(16, 16)) # Un lienzo un poco más grande

    # Layout: Algoritmo de distribución (Spring)
    # k=0.15 controla la distancia entre nodos. Aumentarlo separa más los grupos.
    pos = nx.spring_layout(G, k=0.18, seed=42, iterations=50)

    # Configuración de Colores (Paleta Categórica)
    communities = nx.get_node_attributes(G, 'community_label')
    unique_comms = sorted(list(set(communities.values())))

    # Usamos la paleta "tab10" de Seaborn que tiene colores bien contrastados para categorías
    palette = sns.color_palette("tab10", len(unique_comms))
    color_map = dict(zip(unique_comms, palette))

    # Calculamos el grado (actividad) para definir el tamaño del nodo
    d = dict(G.degree)

    # Dibujamos los nodos iterando por comunidad (para poder hacer la Leyenda)
```

```

for comm in unique_comms:
    # Filtramos qué nodos pertenecen a esta comunidad
    nodelist = [node for node in G.nodes() if communities[node] ==
comm]

    # Definimos tamaño: Base de 30 + (Grado * 15). Los muy activos
serán grandes.
    node_sizes = [d[node] * 15 + 30 for node in nodelist]

    nx.draw_networkx_nodes(G, pos,
                           nodelist=nodelist,
                           node_color=[color_map[comm]], #
Asignamos su color único
                           node_size=node_sizes,
                           alpha=0.85, # Un poco de transparencia
                           label=comm) # <--- ESTO CREA LA
ETIQUETA EN LA LEYENDA

    # Dibujamos las Aristas (Líneas de conexión)
    nx.draw_networkx_edges(G, pos, alpha=0.15, edge_color='gray',
arrows=True)

    # Etiquetas Inteligentes (Solo para los 'Puentes' o Líderes)
    # Usamos Intermediación (Betweenness) para etiquetar a los actores
clave del TP
    betweenness = nx.betweenness centrality(G)
    # Seleccionamos el Top 8 de usuarios más estratégicos
    top_nodes = sorted(betweenness.items(), key=lambda x: x[1],
reverse=True)[:8]

    for node, score in top_nodes:
        x, y = pos[node]
        # Agregamos el texto con un fondo blanco semitransparente para
que se lea bien
        plt.text(x, y+0.025, s=node,
                 bbox=dict(facecolor='white', alpha=0.8,
edgecolor='#cccccc', boxstyle='round,pad=0.2'),
                 horizontalalignment='center', fontsize=10,
fontweight='bold', color='black')

    # Títulos y Leyenda Final
    plt.title("Mapa de Interacciones: Comunidades Ciclistas del AMBA",
fontsize=18, pad=20)
    # Subtítulo explicativo
    plt.suptitle("(Tamaño del nodo = Nivel de Actividad | Color =
Grupo de Origen)",
                 fontsize=12, y=0.89, color='gray')

    # Crear la leyenda

```

```

plt.legend(scatterpoints=1, frameon=True, title="Referencias",
loc='upper right', fontsize=11)

plt.axis('off') # Ocultar ejes X/Y
plt.tight_layout()
plt.show()

plot_network(G)

```

Visualización de las Comunidades Ciclistas del AMBA
(Color = Grupo de Origen)

